

Računarska inteligencija

Minimalna particija
pravougaonika sa unutrašnjim tačkama



Mentor:

dr Vladimir Filipović

Asistent:

Stefan Kapunac

Studenti:

Jana Radojević	184/2019
Marko Petrović	131/2020

Sažetak

Razmatramo problem minimalne particije pravougaonika sa unutrašnjim tačkama (eng. *Minimum Partition of Rectangle with Interior Points*), poznat NP-težak problem računarske geometrije sa primenama u VLSI dizajnu i sečenju materijala. Cilj je uvesti ortogonalne linijske segmente koji dele pravougaonik na manje pravougaonike tako da svaka unutrašnja tačka leži na granici nekog od njih, uz minimizaciju ukupne dužine segmenata. Koristeći redukciju prostora pretrage na Hananovu mrežu, problem svodimo na izbor podskupa ivica mreže. Koristili smo dva algoritma iscrpne pretrage, brute force i branch and bound sa odsecanjem, kao referencu za male instance, kao i tri metaheuristike: simulirano kaljenje i genetski algoritam nad binarnim kodiranjem ivica, te genetski algoritam nad giljotinskim stablom sečenja. Na malim instancama metode upoređujemo sa optimumom iscrpne pretrage, a na većim međusobno. Egzaktni pristup brzo postaje neizvodljiv, dok metaheuristike daju validna rešenja u svim veličinama. Giljotinsko kodiranje značajno nadmašuje binarno na većim instancama jer garantuje validnost svakog rešenja i pretraga znatno manji prostor.

1 Uvod

1.1 Opis problema

Neka je dat pravougaonik R u ravni i konačan skup $P = \{p_1, \dots, p_n\}$ tačaka koje se nalaze strogo u unutrašnjosti R . Tražimo skup međusobno neukrštajućih ortogonalnih (horizontalnih i vertikalnih) linijskih segmenata koji unutrašnjost R dele na manje pravougaonike tako da svaka tačka $p \in P$ leži na granici bar jednog od dobijenih pravougaonika. Ciljna funkcija je ukupna euklidska dužina uvedenih segmenata, koju minimizujemo.

1.2 Motivacija

Problem potiče iz dizajna integrisanih kola visokog stepena integracije (VLSI), gde se prostor čipa deli na pravougane kanale za rutiranje oko predefinisanih pinova ili defekata, pri čemu minimizacija dužine pregrada smanjuje međukanalske smetnje [2]. Srodne primene postoje u sečenju materijala (izolovanje defekata na ivicama pravougaonih komada uz minimalnu dužinu reza) i u arhitektonskom planiranju osnova (floorplanning).

1.3 Računarska složenost

Lingas, Pinter, Rivest i Shamir [10] su dokazali da je problem NP-težak, redukcijom sa problema planarnog 3-SAT-a. Zbog toga ne očekujemo algoritam polinomijalne složenosti koji daje optimalno rešenje, te se prirodno okrećemo aproksimacijama i metaheuristikama.

1.4 Pregled literature

Nakon dokaza NP-težine [10], istraživanja su krenula ka aproksimacijama sa garantovanim faktorom. Gonzalez and Zheng [5] uvode podeli-pa-vladaj aproksimacije, a Du et al. [3] uvode pojam *giljotinske* particije i pokazuju da se optimalna giljotinska particija nalazi dinamičkim programiranjem u $O(n^5)$, uz faktor aproksimacije 2, koji Gonzalez and Zheng [6] poboljšavaju na 1,75. Egzaktno rešavanje za male instance ostvareno je *branch-and-cut* metodom [1, 2].

1.5 Doprinos ovog rada

U skladu sa naučnim karakterom teme, mi *reprodukujemo* postojeće ideje i upoređujemo ih. Konkretno: (i) implementiramo iscrpnu pretragu (bez odsecanja) kao dokazano optimalnu referencu za najmanje instance (2–3 tačke); (ii) implementiramo grananje i ograničavanje sa cenovnim i izvodljivostnim odsecanjem koje proširuje iscrpno rešavanje; (iii) implementiramo dve metaheuristike nad binarnim kodiranjem ivica: simulirano kaljenje [9] i genetski algoritam [4, 8];

i (iv) implementiramo genetski algoritam nad giljotinskim stablom sečenja, čije svako rešenje garantuje validnost particije.

2 Opis rešenja

2.1 Hananova mreža

Kontinualni prostor mogućih segmenata diskretizujemo *Hananovom mrežom* [7]: kroz svaku tačku p_i povučemo horizontalnu i vertikalnu pravu i isečemo ih na granicama R . Skup x -koordinata je $X = \{x_1, \dots, x_n\} \cup \{x_{\min}, x_{\max}\}$, a Y analogno. Preseci ovih pravih čine temena mreže, a susedna temena povezuju ivice. Fundamentalna teorema (videti [3]) tvrdi da uvek postoji optimalno rešenje koje u potpunosti leži na ivicama Hananove mreže, pa problem postaje izbor podskupa unutrašnjih ivica.

2.2 Kodiranje i funkcija cilja

Rešenje predstavljamo binarnim vektorom $x \in \{0, 1\}^m$ nad m unutrašnjih ivica mreže, gde $x_e = 1$ znači da je ivica e izabrana. Ivice na obodu R su uvek prisutne. Funkcija cilja je ukupna dužina izabranih ivica:

$$\min \sum_{e \in E_{\text{int}}} L_e x_e. \quad (1)$$

Izbor je *validna* particija ako: (1) svako teme koje odgovara tački iz P ima interni stepen ≥ 2 (tačka je pokrivena); (2) svako unutrašnje teme koje nije tačka ima stepen 0 ili ≥ 2 (nema visećih segmenata, u literaturi “ostrva” i “kolena” [2]); (3) svi izabrani segmenti su povezani sa obodom R (nema izolovanih ciklusa); i (4) ne postoji teme sa tačno jednom horizontalnom i jednom vertikalnom ivicom (tzv. L-spoj, eng. *L-junction*), osim u četiri ugla pravougaonika R . L-spoj znači da u nekom temenu jedna ivica ide horizontalno a druga vertikalno, što stvara refleksi ugaod od 270° u susednoj regiji, pa ta regija ne bi bila pravougaona. Dozvoljeni stepeni unutrašnjih temena su dakle: 2 (kolinearno prolaz), 3 (T-spoj) i 4 (raskrsnica).

2.3 Iscrpna pretraga (bez odsecanja)

Kao najjednostavniju referencu implementiramo potpunu enumeraciju svih 2^m podskupova unutrašnjih ivica. Za svaki podskup proveravamo validnost i dužinu, i čuvamo najkraće validno rešenje. Ovo je garantovano optimalno, ali je izvodljivo samo za vrlo male instance: za $n = 2$ (12 ivica, $2^{12} = 4096$ podskupova) traje $\sim 0,02$ s, za $n = 3$ (24 ivice, $2^{24} \approx 1,7 \cdot 10^7$ podskupova) ~ 50 s, dok bi za $n = 4$ (40 ivica, $2^{40} \approx 1,1 \cdot 10^{12}$ podskupova) trebalo reda ~ 10 sati, računajući i overhead evaluacije svakog rešenja. Za $n = 5$ (60 ivica, $2^{60} \approx 1,1 \cdot 10^{18}$ podskupova) vreme bi se merilo u hiljadama godina, što je praktično nemoguće. Napomenimo da su instance sa 2–4 tačke premale da bi njihova rešenja bila od praktičnog interesa, ali nam služe kao referencu za proveru ispravnosti ostalih metoda.

```
function Iscrpna(m, ivice):
    najbolja_duzina <- +inf
    for mask in 0 .. 2^m - 1:
        # enumerisemo sve podskupove
        izbor <- bitovi_maske(0/1 po ivici)
        duzina <- suma(L[i] * izbor[i])
        if duzina < najbolja_duzina and validna(izbor):
            najbolja_duzina <- duzina
            najbolji <- izbor
    return najbolji
```

Listing 1: Iscrpna pretraga nad izborom ivica.

2.4 Pohlepna konstrukcija

Pohlepna konstrukcija polazi od svih unutrašnjih ivica izabраниh (što je uvek validna particija), zatim pokušava da ukloni ivice redom od najduže ka najkraćoj, zadržavajući uklanjanje samo ako particija ostaje validna. Ovo daje brzo validno rešenje koje koristimo kao početnu gornju granicu za grananje i ograničavanje, kao i za inicijalizaciju populacije genetskog algoritma.

2.5 Grananje i ograničavanje (eng. branch and bound)

Isti prostor pretrage kao iscrpna enumeracija, ali sa dve vrste odsecanja koje omogućava rešavanje većih instanci:

- **Cenovno odsecanje:** ako tekuća parcijalna dužina dostigne ili premaši najbolju poznatu dužinu, granu napuštamo.
- **Odsecanje izvodljivosti:** ako preostale (neodlučene) ivice ne mogu da zadovolje ograničenja stepena (npr. tačka koja nikako ne može dostići stepen 2, ili neizbežni viseći segment, ili već formiran L-spoj), granu napuštamo.

Početnu gornju granicu daje pohlepno rešenje (opisano u Sekciji 2.4). Grananje ide ivicu po ivicu, pri čemu se prvo bira grana „ne izaberi ivicu”(jeftinija, sondirana prvo da bi se brzo našla dobra rešenja koja stežu granicu).

```
function B&B(i, izbor, duzina):
    if duzina >= najbolja: return                # cenovno odsecanje
    if not moze_zadovoljiti(izbor): return        # odsecanje izvodljivosti
    if i == m:
        if validna(izbor) and duzina < najbolja:
            najbolje <- (izbor, duzina)
        return
    B&B(i+1, izbor + [0], duzina)                # ne biraмо ivicu i
    B&B(i+1, izbor + [1], duzina + L[i])         # biraмо ivicu i
```

Listing 2: Grananje i ograničavanje nad izborom ivica.

2.6 Simulirano kaljenje

Nad istim binarnim kodiranjem optimizujemo funkciju $f(x) = (\sum_e L_e x_e) + \lambda \cdot (\text{broj prekršaja})$, gde kazneni član $\lambda = 1000$ dozvoljava prolazak kroz nevalidan prostor uz podsticaj ka validnosti. Pretraga kreće iz nasumičnog rešenja, čime se izbegava pristrasnost ka jednoj oblasti prostora. Pretraga susedstva koristi tri operatora sa fiksnim verovatnoćama: obrtanje jedne ivice (50%), obrtanje 2–3 ivice (25%), i obrtanje cele mrežne linije, svih ivica na nekom x ili y (25%), čime se poštuje geometrijska struktura. Koristimo geometrijsko hlađenje (faktor 0,9995) i ograničavamo broj iteracija. Za male instance koristimo početnu temperaturu 1000 i 200 000 iteracija, za $n = 30$ temperaturu 1000 i 100 000 iteracija, a za ostale 500 i 20 000. Parametri su odabrani nakon isprobavanja nekoliko različitih vrednosti.

```
function SA(T0, alpha, Tmin, max_iter):
    trenutno <- nasumicni_izbor()                # slucajan binarni vektor
    T <- T0
    najbolje_validno <- None
    while T > Tmin and iter < max_iter:
        sused <- nasumicni_sused(trenutno)       # flip 1 / flip 2-3 / flip liniju
        delta <- f(sused) - f(trenutno)          # f = duzina + lambda * broj_prekršaja
        if delta < 0 or random() < exp(-delta / T):
            trenutno <- sused
```

```

    if validna(trenutno) and duzina(trenutno) < najbolje_validno:
        najbolje_validno <- trenutno
    T <- T * alpha                                # geometrijsko hladjenje
return najbolje_validno

```

Listing 3: Simulirano kaljenje nad binarnim kodiranjem.

2.7 Genetski algoritam (binarno kodiranje)

Hromozom je binarni vektor ivica. Fitnes je $-(\text{dužina} + \text{kazna})$. Koristimo turnirsku selekciju ($k = 3$), kombinaciju uniformnog i dvopozicionog ukrštanja, mutaciju (bit flip uz mutaciju linija koja obrće cele mrežne redove/kolone) i elitizam (top 5 jedinki). Početna populacija meša pohlepna rešenja (25%, kao validna sidra) i nasumična (75%, za raznolikost). Za male instance koristimo populaciju 200 i 1000 generacija, a za ostale populaciju 80 i 200 generacija. Parametri su odabrani nakon isprobavanja različitih vrednosti.

```

function GA(pop_size, num_gen, mut_rate):
    populacija <- 25% pohlepnih + 75% nasumicnih jedinki
    for gen in 1 .. num_gen:
        rangirano <- sortiraj(populacija po fitnesu)    # fitnes = -(duzina + kazna)
        nova_pop <- top 5 (elitizam)
        while |nova_pop| < pop_size:
            roditelj_a <- turnirska_selekcija(k=3)
            roditelj_b <- turnirska_selekcija(k=3)
            (dete_a, dete_b) <- ukrstanje(roditelj_a, roditelj_b) # uniformno ili 2-
tackasto
            nova_pop += [mutacija(dete_a), mutacija(dete_b)]    # bit flip + flip
linije
        populacija <- nova_pop
        azuriraj_najbolje_validno(populacija)
    return najbolje_validno

```

Listing 4: Genetski algoritam nad binarnim kodiranjem ivica.

2.8 Genetski algoritam nad giljotinskim stablom sečenja

Giljotinska particija je particija kod koje svaki rez prolazi kroz ceo tekući pravougaonik, od ivice do ivice. Ovo prirodno definiše *stablo sečenja* (eng. slicing tree): svaki rez deli pravougaonik na dva manja koja se rekurzivno particionuju. Umesto binarnog vektora ivica, hromozom kodira po jedan gen (k_i, o_i) po unutrašnjoj tački, gde k_i određuje izbor pivota u svakoj regiji, a o_i orijentaciju reza. Dekodiranje rekurzivno bira pivot sa najvećim ključem, povlači rez kroz njega i spušta se u dve podregije. Ključna prednost je da je svako rešenje automatski validna particija, pa kazneni član nije potreban, a prostor pretrage je nad n tačaka umesto nad m ivica, što je eksponencijalno manje. Operatori (selekcija, ukrštanje, mutacija) su analogni kao kod binarnog GA. Cena je ograničenje na giljotinske particije, koje mogu biti duže od optimalnih, ali kako pokazuju rezultati, na većim instancama ovaj pristup značajno nadmašuje binarno kodiranje.

3 Eksperimentalni rezultati

3.1 Test instance

Pravougaonik je dimenzija 100×100 . Generišemo tri tipa instanci: *nasumične* (sve koordinate različite, nekorektilinearni), *klasterovane*, i sa *deljenim koordinatama*.

3.2 Male instance: poređenje sa optimumom iscrpne pretrage

Tabela 1 prikazuje dužine rešenja za nasumične instance sa 2 i 3 tačke, gde je optimum dostupan i iz iscrpne pretrage i iz B&B, i za 4 tačke gde samo B&B dostiže optimum. Za $n = 2$ i $n = 3$ metaheuristike koriste pojačane parametre (SA: 200 000 iteracija, GA i GA-g: populacija 200, 1000 generacija) da bi sigurno dostigle optimum.

Tabela 1: Male nasumične instance: dužine rešenja i vreme iscrpne pretrage. Podebljano: dokazano optimalno.

n	ivica	Iscrpna	$t_{isc}[s]$	B&B	$t_{BB}[s]$	SA	GA	GA-g
2	12	130	0,02	130	0,005	130	130	130
3	24	178	50,6	178	0,86	178	178	178
4	40	—	—	210	204	243	210	210

Diskusija. Iscrpna pretraga i B&B daju identične optimume za $n = 2$ i $n = 3$, što potvrđuje ispravnost oba implementirana rešavača. Za $n = 3$, iscrpna pretraga pregleda $2^{24} \approx 1,7 \cdot 10^7$ podskupova za ~ 50 s, dok B&B zahvaljujući odsecanju pronalazi optimum za 0,86 s, što je ubrzanje od $\sim 58\times$. Za $n = 4$ (40 ivica), iscrpna pretraga bi zahtevala $\sim 1,1 \cdot 10^{12}$ podskupova (procenjeno ~ 10 sati), dok B&B završava za ~ 204 s.

Za $n = 2$, sve tri metaheuristike pronalaze optimalno rešenje (130). Za $n = 3$, sa pojačanim parametrima, SA, GA i GA-g takođe dostižu optimum (178). Za $n = 4$, sa standardnim parametrima, GA i GA-g dostižu optimum (210), dok SA zaostaje (243).

3.3 Srednje instance: međusobno poređenje metaheuristika

Za $n \geq 8$ egzaktni optimum više nije praktično dostupan, pa metaheuristike poredimo međusobno. Tabela 2 prikazuje rezultate za $n = 8$ i $n = 15$ sa standardnim parametrima (SA: 20 000 iteracija, GA i GA-g: 200 generacija, populacija 80).

Tabela 2: Srednje nasumične instance. SA: 20 000 iteracija, GA i GA-g: 200 generacija, populacija 80. Podebljano: najkraće rešenje.

n	ivica	SA	GA	GA-g
8	144	650	703	382
15	480	1333	1116	546

Diskusija. Za $n = 8$ (144 ivice), GA-g nadmašuje SA i GA (382 vs. 650 vs. 703), dok za $n = 15$ (480 ivica) GA-g daje 546 naspram 1116 (GA) i 1333 (SA). Razlika je izrazita i raste sa brojem tačaka: binarno kodiranje pretražuje prostor $\{0,1\}^m$ koji eksponencijalno raste sa brojem ivica, dok GA-g pretražuje samo prostor giljotinskih stabala nad n tačaka, gde je svako rešenje garantovano validno. SA sa 20 000 iteracija dovoljno dobro istražuje prostor za $n = 8$, ali za $n = 15$ prostor $\{0,1\}^{480}$ postaje prevelik za nasumične poteze. GA-g, iako ograničeno na giljotinske particije, značajno bolje skalira jer je njegov prostor pretrage reda veličine $n!$ umesto 2^m .

3.4 Velika instanca

Da bismo ispitali ponašanje na velikim instancama, pokrenuli smo metaheuristike na instanci sa $n = 30$ (1860 ivica). SA koristi 100 000 iteracija (početna temperatura 1000), GA i GA-g koriste 200 generacija sa populacijom 80. Rezultati su u Tabeli 3.

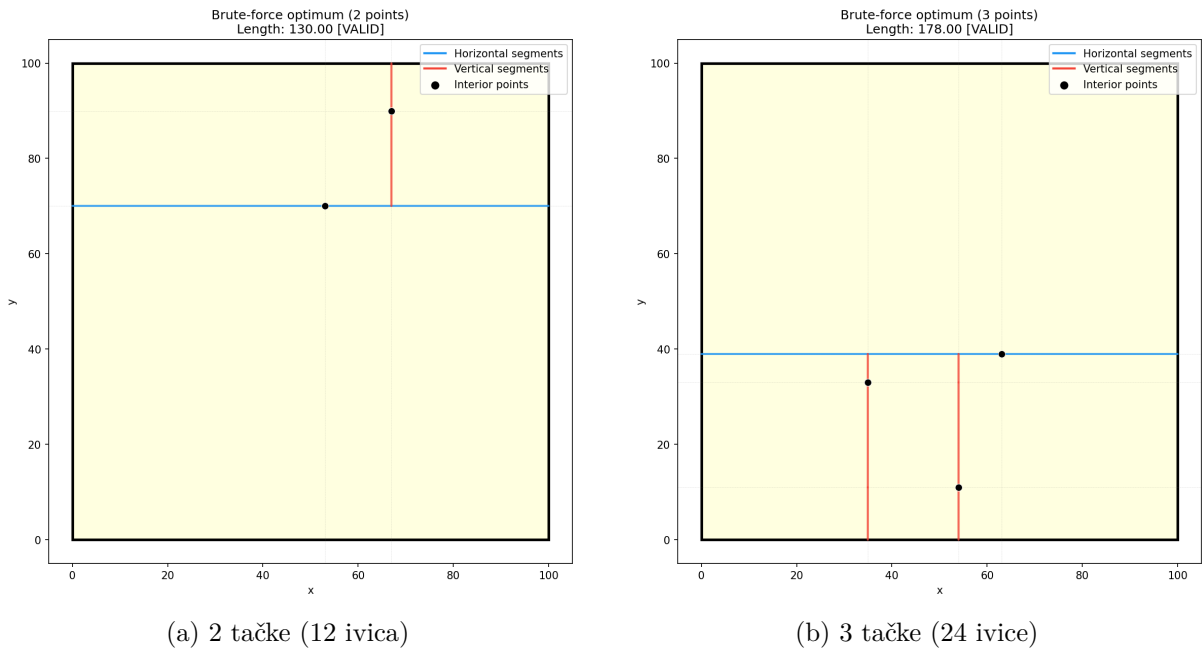
Tabela 3: Velika instanca ($n = 30$, 1860 ivica). Podebljano: najkraće rešenje.

Metoda	Dužina	Vreme [s]
SA (100k iteracija)	3428	62,5
GA (200 gen, pop 80)	2660	43,6
GA-g (200 gen, pop 80)	875	48,7

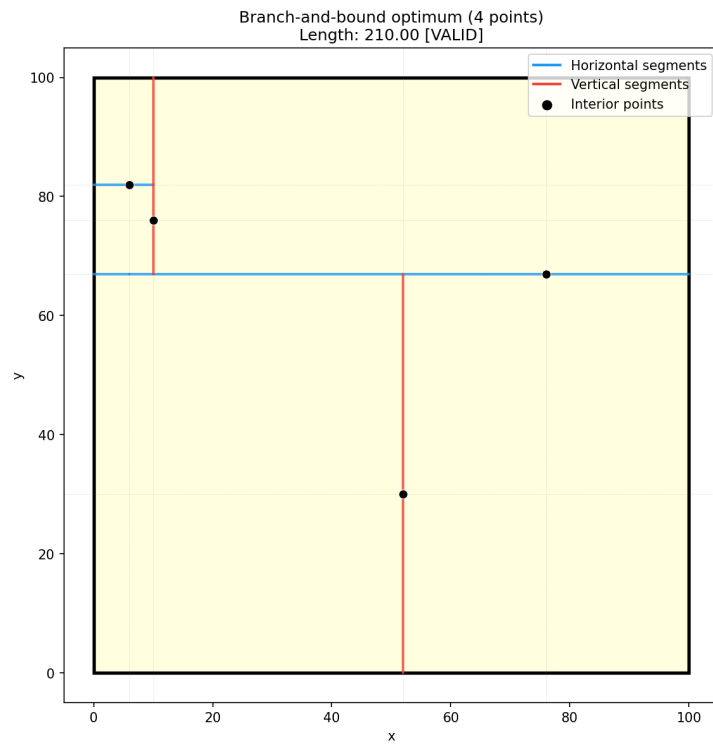
Diskusija. Na $n = 30$, GA-g daje 875, što je $3\times$ bolje od GA (2660) i $4\times$ bolje od SA (3428). Binarno kodiranje ne uspeva da pronadje kvalitetna rešenja na ovoj veličini: prostor $\{0, 1\}^{1860}$ je veličine $2^{1860} \approx 10^{560}$, što je nedostižno za bilo koji budžet. GA-g, iako ograničeno na giljotinske particije, uspeva jer je njegov prostor pretrage znatno manji i svako rešenje je validno. Ovo potvrđuje da izbor kodiranja ima ključan uticaj na kvalitet metaheurističke pretrage.

3.5 Vizualizacija

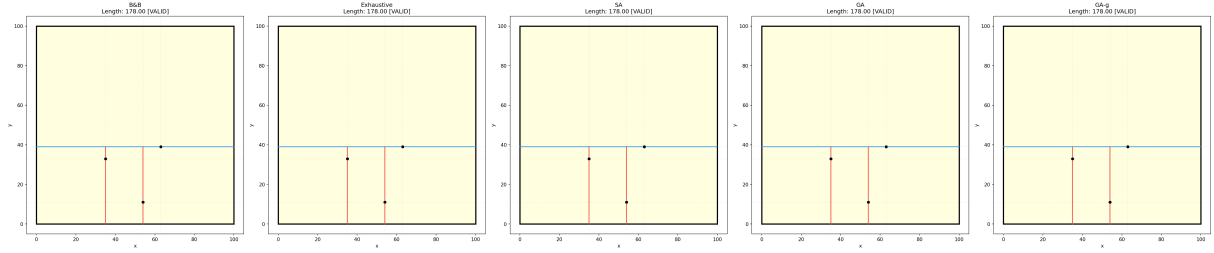
Na malim, vizualizovanim instancama proveravamo geometrijsku korektnost rešenja. Slika 1 prikazuje optimum dobijen iscrpnom pretragom za 2 i 3 tačke, Slika 2 optimum B&B za 4 tačke, Slika 3 uporedni prikaz svih metoda na instancama sa 3, 8, 15 i 30 tačaka, a Slika 4 konvergenciju SA i GA.



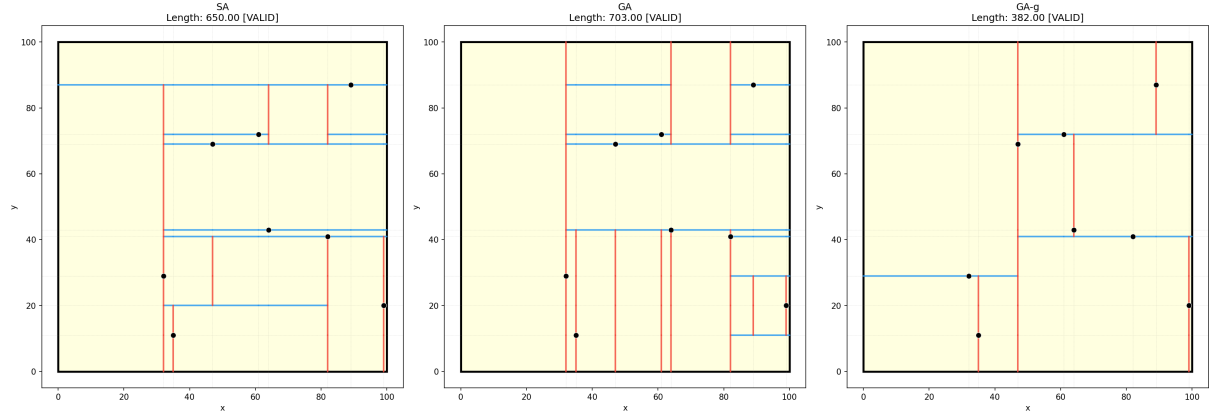
Slika 1: Optimalne particije dobijene iscrpnom pretragom (bez odsecanja). Plavo: horizontalni segmenti, crveno: vertikalni.



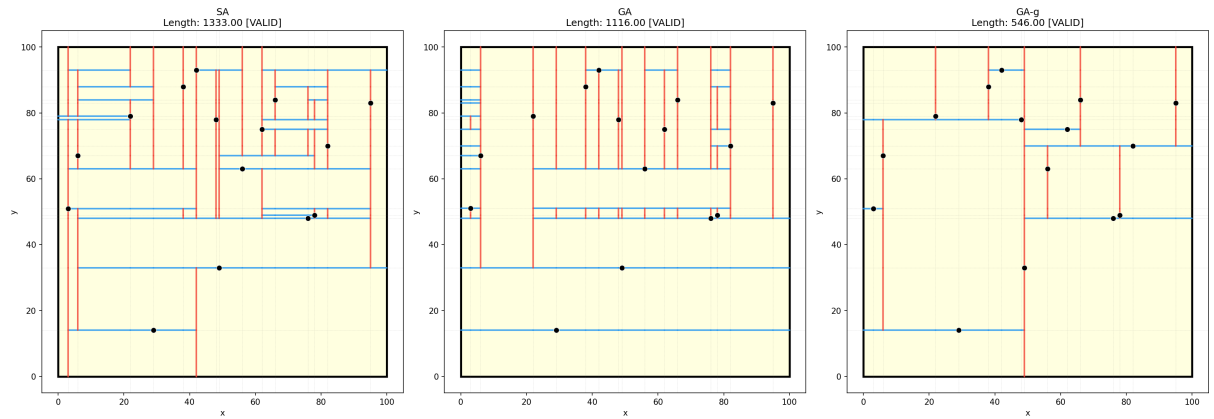
Slika 2: Optimalna particija dobijena grananjem i ograničavanjem za 4 tačke (40 ivica, ~ 204 s).
Iscrpna pretraga bi zahtevala $\sim 1,1 \cdot 10^{12}$ podskupova.



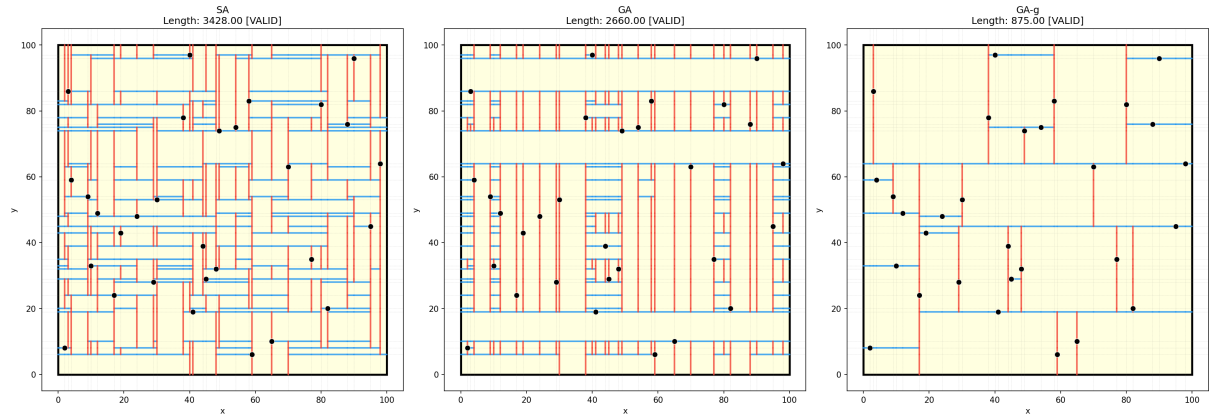
(a) 3 tačke



(b) 8 tačaka

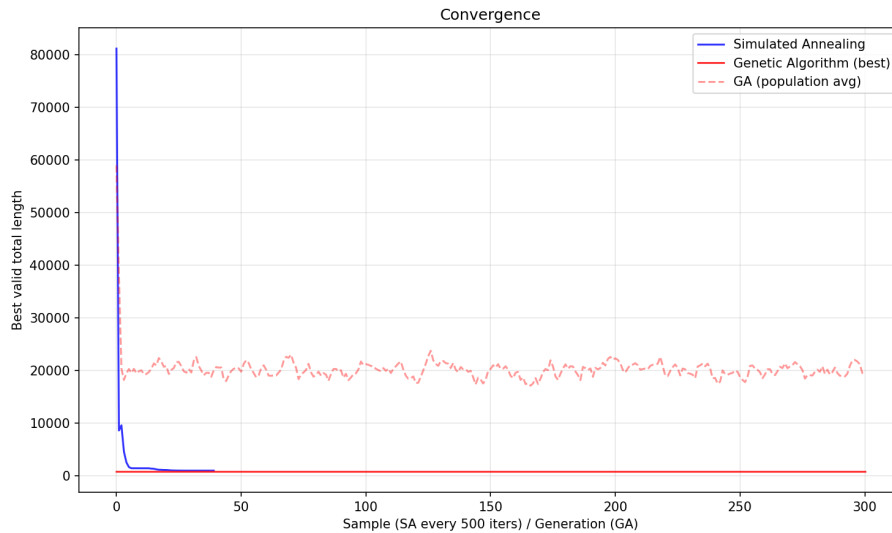


(c) 15 tačaka



(d) 30 tačaka

Slika 3: Upoređenje svih metoda na instancama sa 3, 8, 15 i 30 tačaka. Na malim instancama B&B daje optimum; na većim metaheuristike se međusobno poredimo. GA-g konzistentno daje najkraća rešenja za $n \geq 8$.



Slika 4: Konvergencija simuliranog kaljenja i genetskog algoritma (najbolja validna dužina tokom iteracija/generacija) na instanci sa 12 tačaka.

3.6 Objektivnost rezultata

Naše metaheuristike *nisu* bolje od egzaktnih metoda ni od najboljih aproksimacija iz literature; one služe da pokažu da se validna rešenja mogu dobiti u izvodljivom vremenu i na instancama gde egzaktni pristup nije primenljiv. Ključni praktični nalaz je da izbor kodiranja ima presudan uticaj: binarno kodiranje nad ivicama Hananove mreže brzo postaje neupotrebljivo zbog eksponencijalnog rasta prostora $\{0, 1\}^m$, dok giljotinsko stablo sečenja znatno bolje skalira jer je prostor pretrage manji i svako rešenje je validno.

4 Zaključak

Reprodukovali smo i uporedili više pristupa problemu minimalne particije pravougaonika sa unutrašnjim tačkama: iscrpnu pretragu i grananje i ograničavanje kao egzaktne metode, te tri metaheuristike: simulirano kaljenje i genetski algoritam nad binarnim kodiranjem, i genetski algoritam nad giljotinskim stablom sečenja. Eksperimenti potvrđuju teorijska očekivanja: egzaktni pristup je upotrebljiv samo za najmanje instance (iscrpna pretraga do 3 tačke, B&B do 4), dok metaheuristike daju validna rešenja u svim veličinama. Binarno kodiranje brzo degradira sa rastom broja tačaka: za $n = 15$ (480 ivica) SA i GA daju rešenja reda 1000, dok GA-g daje 546. Za $n = 30$ (1860 ivica) razlika je još izrazitija: GA-g daje 875 naspram 2660 (GA) i 3428 (SA). Giljotinsko kodiranje, iako ograničeno na particije gde svaki rez prolazi kroz ceo pravougaonik, znatno bolje skalira jer je prostor pretrage nad n tačaka umesto nad m ivicama, i svako rešenje je garantovano validno. Pravci daljeg rada uključuju egzaktnu giljotinsku DP referencu $O(n^5)$ [3], hibridizaciju binarnog i giljotinskog kodiranja, i lokalnu pretragu (tabu).

Literatura

- [1] Felipe C. Calheiros, Abilio Lucena, and Cid C. de Souza. Optimal rectangular partitions. *Networks*, 41(1):51–67, 2003. doi: 10.1002/net.10058.
- [2] Cláudio N. de Meneses and Cid C. de Souza. Exact solutions of rectangular partitions via integer programming. *International Journal of Computational Geometry & Applications*, 10(5):477–522, 2000. doi: 10.1142/S0218195900000280.

- [3] Ding-Zhu Du, L. Q. Pan, and Man-Tak Shing. Minimum edge length guillotine rectangular partition. Technical Report 02418-86, Mathematical Sciences Research Institute, University of California, Berkeley, 1986.
- [4] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [5] Teofilo F. Gonzalez and Si-Qing Zheng. Bounds for partitioning rectilinear polygons. In *Proceedings of the 1st Annual Symposium on Computational Geometry (SoCG)*, pages 281–287, 1985. doi: 10.1145/323233.323269.
- [6] Teofilo F. Gonzalez and Si-Qing Zheng. Improved bounds for rectangular and guillotine partitions. *Journal of Symbolic Computation*, 7(6):591–610, 1989. doi: 10.1016/S0747-7171(89)80042-2.
- [7] Maurice Hanan. On Steiner’s problem with rectilinear distance. *SIAM Journal on Applied Mathematics*, 14(2):255–265, 1966. doi: 10.1137/0114025.
- [8] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.
- [9] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.
- [10] Andrzej Lingas, Ron Y. Pinter, Ronald L. Rivest, and Adi Shamir. Minimum edge length partitioning of rectilinear polygons. In *Proceedings of the 20th Annual Allerton Conference on Communication, Control, and Computing*, pages 53–63, Monticello, Illinois, 1982.